

Surplus-to-Shelter: Real-Time Food Rescue Network

GitHub Repository Specification & Documentation * <https://github.com/J9079/Hemi-Hacks>

Repository: github.com/J9079/Hemi-Hacks	Runtime: Node.js v20+ / Express 4	Frontend: React 18 / Vite / Tailwind
Database: MongoDB Atlas Cloud Cluster	Real-Time: Socket.IO Mesh Channels	Deploy: Vercel Edge + Render Web Svc

1. Executive Summary & Problem Solved

Globally, more than one-third of all food produced is wasted, generating approximately **8-10% of global greenhouse gas emissions** while local community shelters face recurring food deficits. Traditional rescue initiatives rely on disjointed phone calls, manual spreadsheets, and ad-hoc chat groups. By the time logistical coordination takes place, perishable food spoils.

Surplus-to-Shelter replaces manual coordination with an automated, capacity-aware logistics engine that connects commercial donors with nearby verified shelters and volunteer drivers through real-time algorithmic matching and turnkey dispatch.

2. System Architecture & 4-Tier Blueprint

- **Presentation Layer:** React 18 Single Page Application (SPA), Vite bundler, Tailwind CSS, OpenStreetMap Leaflet Engine, and Web Audio API synthesizer for dispatch chimes.
- **API Gateway & Transport:** Express 4 server hosting RESTful API controllers, JWT authentication guards, RBAC authorization, and Socket.IO real-time event broadcasting. Statically serves compiled frontend (client/dist) with SPA wildcard fallback.
- **Core Domain Engines:** Multi-Factor Shelter Matching Engine (Haversine proximity, shelter capacity, expiry buffer), Delivery State Machine (FSM), and Landfill Methane Abatement Carbon Calculator.
- **Data & Cloud Persistence:** MongoDB Atlas cloud replica set cluster (cluster0.i2iv3xy.mongodb.net), Mongoose schemas with 2dsphere geospatial indexing, and serverless connection pooling.

3. 4 Dedicated Role-Based Portals

Portal	Target Audience	Key Operational Capabilities
Food Donor	Restaurants, Caterers, Hotels, Cafeterias	Post surplus food wizard; auto-detect GPS coordinates; live expiry simulator; full CRUD to update or cancel active posts before dispatch.
NGO / Shelter	Shelters, Food Banks, Community Kitchens	Real-time available capacity intake slider; match candidate radar; compatibility score breakdown; 1-click accept or justified reject.
Volunteer Driver	Two-wheelers, Vans, Citizen Volunteers	Nearest rescue radar sorted by Haversine km; Web Audio chime alert; turn-by-turn state progression; proximity arrival detector.
Admin Command	Logistics Coordinators & Auditors	Interactive OpenStreetMap Leaflet mesh; live dynamic meals rescued and CO2e avoided audit counters; user management & RBAC controls.

4. Algorithmic Matching Scoring Engine

Candidate shelters within a 15 km radius are scored using a multi-factor weighted algorithm to determine the optimal shelter for each donation:

Match Score = $(0.35 * S_{dist}) + (0.30 * S_{cap}) + (0.20 * S_{urg}) + (0.15 * S_{exp})$

Factor	Weight	Mathematical Logic	Logistical Impact
Distance (S_{dist})	35%	1 - (Distance / 15 km)	Rewards close proximity to minimize volunteer transit time.
Capacity (S_{cap})	30%	Incoming Meals / Available Capacity	Prevents shelter food overflow and validates storage room.
Urgency Need (S_{urg})	20%	Critical=1.0, High=0.8, Med=0.5	Prioritizes shelters experiencing severe food shortages.
Transit Safety (S_{exp})	15%	(Hours Left - Transit) / Hours Left	Rejects matches if food would spoil during transit.

5. Delivery State Machine & Environmental ESG Model

Delivery Lifecycle FSM: [POSTED] ➡ [MATCHED] ➡ [ASSIGNED] ➡ [PICKUP_STARTED] ➡ [PICKED_UP] ➡ [DELIVERED]

- **Meal Conversion Benchmark:** 1 Meal = 0.42 kg edible food (United Nations FAO global standard).
- **Methane Avoidance Factor:** Landfill organic decomposition produces potent methane (CH4). Every 1 kg of edible food rescued avoids **2.5 kg of CO2e emissions**.
- **Formula:** CO2e Avoided (kg) = Food Rescued (kg) * 2.5. Computed automatically on delivery confirmation.

6. RESTful API Reference Summary

Method & Route	Access	Description
POST /api/auth/register	Public	Register new Donor, NGO, or Driver with GPS coordinates.
POST /api/auth/login	Public	Authenticate user and return signed JWT Bearer token.
PUT /api/auth/profile	Protected	Update personal info, phone, address, coordinates, capacity.
POST /api/donations	Donor, Admin	Post new surplus rescue; triggers automated matching engine.
PUT /api/donations/:id	Donor, Admin	Edit meal count, packaging, temperature notes, expiry time.
DELETE /api/donations/:id	Donor, Admin	Cancel/delete unassigned donation post.
PUT /api/ngos/:id/capacity	NGO, Admin	Update intake capacity and available meal thresholds.
POST /api/ngos/donations/:id/accept	NGO, Admin	Accept matching surplus food; dispatches driver alert.
POST /api/drivers/requests/:id/accept	Driver, Admin	Claim dispatch request and initialize active delivery route.
PUT /api/deliveries/:id/status	Driver, Admin	Transition delivery state (PICKED_UP, DELIVERED).
GET /api/dashboard/public-stats	Public	Live community impact metrics for unauthenticated landing page.

7. Local Setup & Cloud Deployment Guide

One-Command Production Build & Run:

```
# 1. Clone repository git clone https://github.com/J9079/Hemi-Hacks.git cd Hemi-Hacks # 2. Build React client into client/dist and install dependencies npm run build # 3. Start unified All-in-One server on port 5000 npm start # Application active at http://localhost:5000 (serves SPA + APIs) # Health check: http://localhost:5000/api/health
```

Cloud Deployment Options:

Platform	Configuration	Deployment Workflow
Vercel (Edge & Serverless)	vercel.json api/index.js	1. Import J9079/Hemi-Hacks in Vercel. 2. Build Command: npm run build, Output: client/dist. 3. Add environment variables (MONGODB_URI, JWT_SECRET). 4. Deploy: Global Edge CDN for UI + Serverless API functions.
Render (Persistent Server)	render.yaml	1. Create Web Service on Render connecting GitHub repo. 2. Build: npm run build, Start: npm start. 3. Single-port server provides native WebSockets for Socket.IO. 4. Connects to MongoDB Atlas cluster with automatic HTTPS.

8. Project Directory Structure

```
Hemi-Hacks/ ■■■ api/ ■■■ index.js # Vercel serverless entry point ■■■ client/ # React 18 SPA Frontend (Vite + Tailwind) ■■■ dist/ # Compiled production build ■■■ src/ ■■■ components/ # Reusable UI & Leaflet RescueMap ■■■ context/ # AuthContext & SocketContext (Audio chime) ■■■ pages/ # Donor, NGO, Driver, Admin, Profile ■■■ App.jsx # Protected role-based routing ■■■ package.json ■■■ server/ # Express Backend & Algorithmic Engines ■■■ config/ # MongoDB Atlas connection pooling & constants ■■■ controllers/ # REST controllers (Full CRUD across all models) ■■■ models/ # Mongoose Schemas (User, Donation, Delivery) ■■■ routes/ # Express API routers ■■■ services/ # Matching engine & impact calculator ■■■ server.js # Unified Express server (Socket.IO + client/dist) ■■■ render.yaml # Render Blueprint deployment specification ■■■ vercel.json # Vercel Edge & Serverless configuration ■■■ README.md # Platform documentation
```